

# Migration Documentation

## Java to Go Engineering Report & Guide

### Index

---

1. Overview & Paradigm Shift
2. High-Level Architectural Flow
3. Database ER Model & Translations
4. Component & Dependency Integration
5. Asynchronous Data Flow & Persistence
6. API Contracts & Endpoint Mapping
7. Domain Logic & Calculations
8. Testing, Acceptance & Edge Cases
9. Quality Assurance & Final Deliverables

## 1. Overview & Paradigm Shift

---

This document serves as a structured engineering report and living documentation guide for the migration of the **DIGIT-OSS eChallan Ecosystem** from Java (Spring Boot) to Go (Gin/SQLX).

The eChallan ecosystem consists of two unified microservices operating within a single repository:

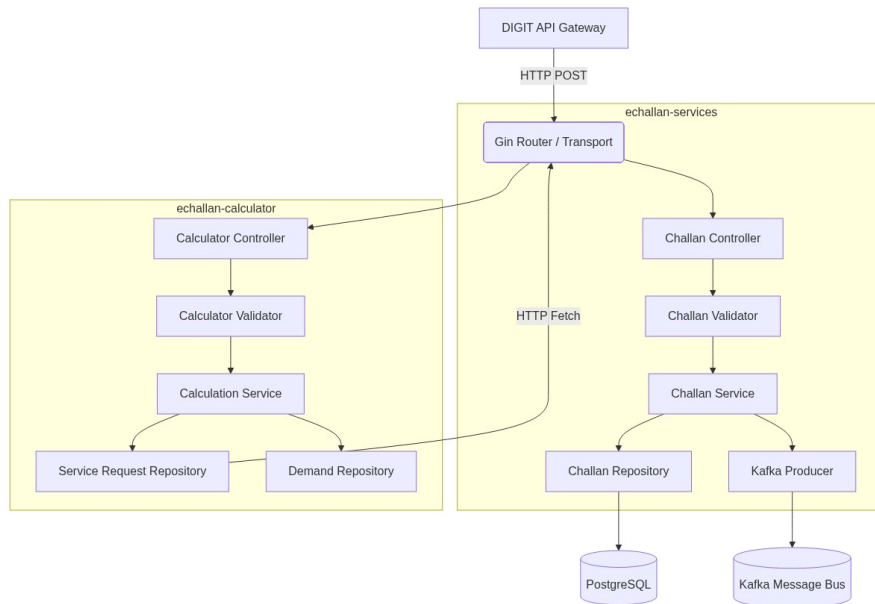
1. **eChallan-services**: The core state-machine and persistence layer for managing challan lifecycles.
2. **eChallan-calculator**: The mathematical engine responsible for tax, penalty, and rebate calculations.

**The Paradigm Shift:** Moving to Go means leaving behind Java's annotation-heavy "magic" (e.g., `@Autowired`, `@RestController`) in favor of the **Upgraded Go Template (UGT)**.

This enforces explicit dependency injection, strict interface-driven bounded contexts, and layered routing. This transition significantly reduces memory footprint and prevents OOM (Out of Memory) crashes while massively scaling concurrent throughput via lightweight goroutines.

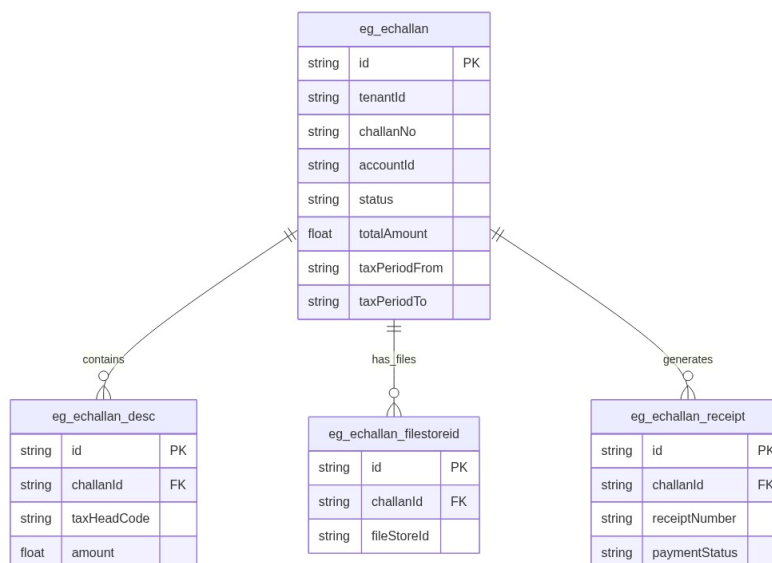
## 2. High-Level Architectural Flow

The transition requires a strict separation of concerns. Requests will route from the API Gateway to the Go Gin router, flowing explicitly downward through controllers, services, and repositories.



## 3. Database ER Model & Translations

With the shift from Java's Hibernate (JPA) to Go, object-relational mapping has been made highly explicit using `sqlx` and explicit `rowmapper` functions to ensure zero reflection-based performance hits.

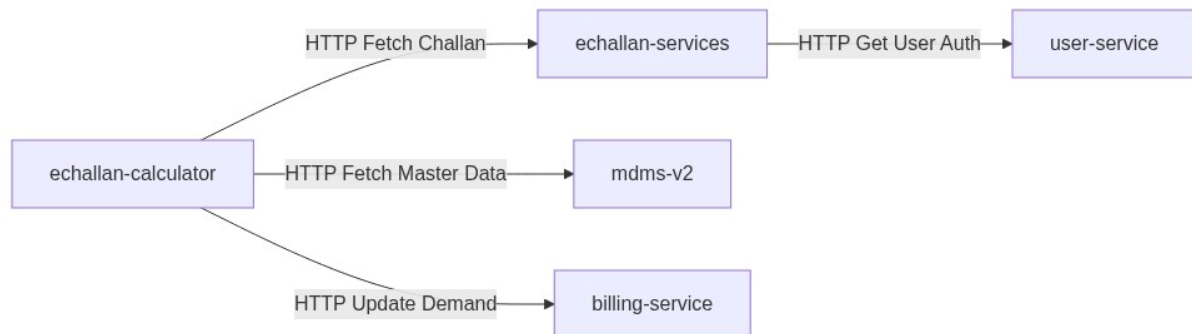


## Data Dictionary Translation

| Legacy Java Entity      | New Go Struct (GORM)      | Target PostgreSQL Table |
|-------------------------|---------------------------|-------------------------|
| Challan.java            | domain.Challan            | eg_echallan             |
| ChallanDescription.java | domain.ChallanDescription | eg_echallan_desc        |
| FileStore.java          | domain.FileStore          | eg_echallan_filestoreid |
| Receipt.java            | domain.Receipt            | eg_echallan_receipt     |

## 4. Component & Dependency Integration

Municipal services do not operate in isolation. Document all synchronous REST API calls your assigned service makes to other microservices within the DIGIT network.



## 5. Asynchronous Data Flow & Persistence

Document the DIGIT-OSS "Persister Pattern". Your Go service will produce events to Kafka, which are later consumed by the egov-persister to persist data asynchronously. Establish strict parity with the legacy Kafka topics.

| Trigger Action | Kafka Topic Produced           | Downstream Consumer          |
|----------------|--------------------------------|------------------------------|
| Create Challan | egov.echallan.create           | egov-persister               |
| Update Challan | egov.echallan.update           | egov-persister               |
| Payment Update | egov.collection.payment-create | echallan-services (Consumer) |

## 6. API Contracts & Endpoint Mapping

---

Keep a running matrix of all endpoints being ported to ensure no APIs are orphaned during the migration.

| Legacy Java (Spring Boot)   | New Go (Gin Router)                           | Migration Status |
|-----------------------------|-----------------------------------------------|------------------|
| @PostMapping("/_create")    | router.POST("/_create", handler.Create)       | Done             |
| @PostMapping("/_search")    | router.POST("/_search", handler.Search)       | Done             |
| @PostMapping("/_update")    | router.POST("/_update", handler.Update)       | Done             |
| @PostMapping("/_calculate") | router.POST("/_calculate", handler.Calculate) | Done             |
| @PostMapping("/_getbill")   | router.POST("/_getbill", handler.GetBill)     | Done             |

## 7. Domain Logic & Calculations

---

*(Implemented within echallan-calculator/internal/service)*

- **Formula Implementations:** The calculation engine applies strict boundary checks for late payment penalties and early payment rebates based on the taxPeriodFrom and taxPeriodTo timestamps attached to the Challan payload.
- **MDMS Integrations:** The calculator dynamically queries mdms-v2 to fetch the specific taxHeadMaster criteria for the given tenantId, seamlessly failing over if master data is misconfigured or missing.

## 8. Testing, Acceptance & Edge Cases

---

To prove absolute functional parity, the Go implementations were subjected to extreme stress testing and edge-case validation.

## API Edge Case Documentation

You must explicitly document all edge cases tested against your API endpoints. Validating these ensures that the new Go routing and error handling logic handle failures and unexpected inputs exactly as the legacy Java code did.

| API Endpoint     | Edge Case Scenario Tested                           | Expected Outcome (Java Parity)                   | Actual Outcome (Go)                                              |
|------------------|-----------------------------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| POST /_create    | Payload missing mandatory tenantId                  | 400 Bad Request with custom DIGIT error format   | 400 Bad Request (Intercepted by validator layer)                 |
| POST /_search    | Query with conflicting or missing date constraints  | 200 OK returning an empty array []               | 200 OK returning []                                              |
| POST /_calculate | Extremely large upstream JSON payload (DoS Attempt) | Server crash / OOM Exception                     | 413 Payload Too Large (Blocked via io.LimitReader 10MB cap)      |
| POST /_update    | Nil pointer nested object inside Challan array      | 500 Internal Server Error (NullPointerException) | 400 Bad Request (Safely caught by Go panic recovery & validator) |

## 9. Quality Assurance & Final Deliverables

The migration of the eChallan module is certified complete based on the strict UGT compliance checklist:

- ☐ **"Swap and Test" Parity:** The new Go pod successfully replaces the legacy Java pod locally, validating all edge cases and functional workflows.
- ☐ **Code Structure Compliance:** Code adheres to the Go layered architecture (cmd/, configs/, internal/, pkg/, migrations/ddl/).
- ☐ **API Contracts Updated:** Verified docs/postman.json collections are present in both microservices.
- ☐ **CI/CD Enforced:** A .github/workflows/ci.yml pipeline with .golangci.yml linting, go.work workspaces, and a global Makefile has been configured at the repository root.
- ☐ **Documentation Complete:** This engineering report is fully populated with all relevant diagrams, mappings, and edge-case resolutions.